```
(*
                              CS51 Lab 12
                  Imperative Programming and References
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 15 of the textbook
    <http://book.cs51.io>, which should be read before the lab session.

Objective:

    This lab provides practice with reference types and their use in
    building mutable data structures and in imperative programming more
    generally. It also gives further practice in using modules to
    implement abstract data types.

    There are three total parts to this lab. Please refer to the following
    files to complete all exercises:

    -> lab12_part1.ml -- Part 1: Fun with references (including `gensym`)
       lab12_part2.ml -- Part 2: Appending mutable lists
       lab12_part3.ml -- Part 3: Adding serialization to imperative queues
======================================================================*)

(*======================================================================
                              IMPORTANT

As usual, when implementing functions in this lab, you shouldn't feel
beholden to how the definition is introduced in the skeleton code
below. For instance, you might want to add a `rec`, or use a different
argument list, or no argument list at all but binding to an anonymous
function instead. THIS IS ESPECIALLY PERTINENT TO CERTAIN PROBLEMS IN
THIS LAB, WHERE THE NORMAL COMPACT NOTATION FOR INTRODUCING FUNCTIONS
MAY NOT BE APPROPRIATE.
 ******************************************************************)

(*======================================================================
Part 1: Fun with references (including `gensym`)

......................................................................
Exercise 1: Consider a function `incr` that takes an `int ref`
argument and has the side effect of incrementing the integer stored in
the `ref`. What is an appropriate type for the return value? What
should the type for the function as a whole be?
......................................................................*)

(* Know the answer? Call over a staff member to check. If you're
working on this lab outside of lab times, check
<https://url.cs51.io/lab12-1> for our answer, which you should keep to
in the next exercise. *)
```

```
(*.....................................................................
Exercise 2: Now implement the `incr` function.
.....................................................................*)

let incr _ =
  failwith "incr not implemented" ;;

(*.....................................................................
Exercise 3: Write a function `remember` that returns the last string
that it was called with. The first time it is called, it should return
the empty string.

    # remember "don't forget" ;;
    - : string = ""
    # remember "what was that again?" ;;
    - : string = "don't forget"
    # remember "etaoin shrdlu" ;;
    - : string = "what was that again?"

(This is probably the least "functional" function ever written!)

As usual, you shouldn't feel beholden to how the definition is
introduced in the skeleton code below.
.....................................................................*)

let remember _ =
  failwith "remember not implemented" ;;

(* The `gensym` function (short for "GENerate SYMbol") has a long
history dating back to the early days of the programming language
LISP.  You can find it as early as the 1974 MacLISP manual (page
53). <https://url.cs51.io/lisp-gensym>

    (What is LISP you ask? LISP is an untyped functional programming
    language invented by John McCarthy in 1958, which he based
    directly on Alonzo Church's lambda calculus. It is one of the most
    influential programming languages ever devised. You could do worse
    than spend some time learning the Scheme dialect of LISP, which,
    by the way, will be made much easier by having learned a typed
    functional language -- OCaml.)

The `gensym` function takes a string and generates a new string by
suffixing a unique number, which is initially 0 but is incremented
each time `gensym` is called.

For example,

    # gensym "x" ;;
    - : string = "x0"
    # gensym "x" ;;
    - : string = "x1"
    # gensym "something" ;;
    - : string = "something2"
```

```
    # gensym (gensym "x") ;;
    - : string = "x34"

There are many uses of `gensym`, one of which is to generate new
unique variable names in an interpreter for a programming language. In
fact, you'll need `gensym` for just this purpose in completing the
final project for CS51, so this is definitely not wasted effort.

.....................................................................
Exercise 4: Complete the implementation of `gensym`. As usual, you
shouldn't feel beholden to how the definition is introduced in the
skeleton code below. (We'll stop mentioning this from now on.)
.....................................................................*)

let gensym (s : string) : string =
  failwith "gensym not implemented" ;;
```